Desafío II: Pre-informe de Análisis y Diseño de Ude@Book

Paulina Quintero Arenas, Luis Castillo Chicaiza

Facultad de Ingeniería Universidad de Antioquia
Informatica II

1. Análisis del problema

El reto principal de Ude@Book radica en la gestión de una gran cantidad de datos (mínimo 1000 usuarios) bajo la restricción estricta de no utilizar contenedores de la STL ni herencia, garantizando además eficiencia en tiempo y memoria.

Para hacer esto posible, nos vamos a enfocar en armar nuestras propias estructuras dinámicas. Además, en vez de estar copiando los datos de un lado a otro, vamos a relacionar todo por medio de **índices**, garantizando que la información original no se duplique y ocupe menos memoria.

2. Decisiones de diseño

A continuación, presentamos las decisiones/análisis para resolver los retos del sistema:

- **Gestión de Colecciones:** Implementaremos una plantilla basada en arreglos dinámicos que duplican su capacidad automáticamente similar a la del desafío I. Elegimos arreglos en lugar de listas porque necesitamos acceso directo por posición para funciones como las fotos y la búsqueda binaria.
- **Representación de Amistades:** se construirán como un único arreglo lineal de tamaño $n \times n$ (donde n es la cantidad de usuarios), accediendo a la posición matemática `[fila * n + columna]`.
- **Almacenamiento de Mensajes:** Evitaremos guardar el texto de un mensaje dos veces (una en el que lo envía y otra en el que recibe). Se creará un conjunto global de mensajes en la clase controladora (`Plataforma`), y cada usuario solo guardará un vector con los índices referenciando a los mensajes que le pertenecen.
- **Sugerencias de Amistad:** Para evitar mantener datos desactualizados, las sugerencias no se guardarán en el perfil del usuario. Se calcularán en tiempo real al solicitarlas.
- **Búsqueda Rápida de Usuarios:** Mantendremos el arreglo principal de usuarios en orden alfabético al cargarlo desde los archivos. Esto nos permite usar búsqueda Binaria para encontrar cualquier usuario en lugar de recorrer todo secuencialmente.

2.1. Formato de archivos

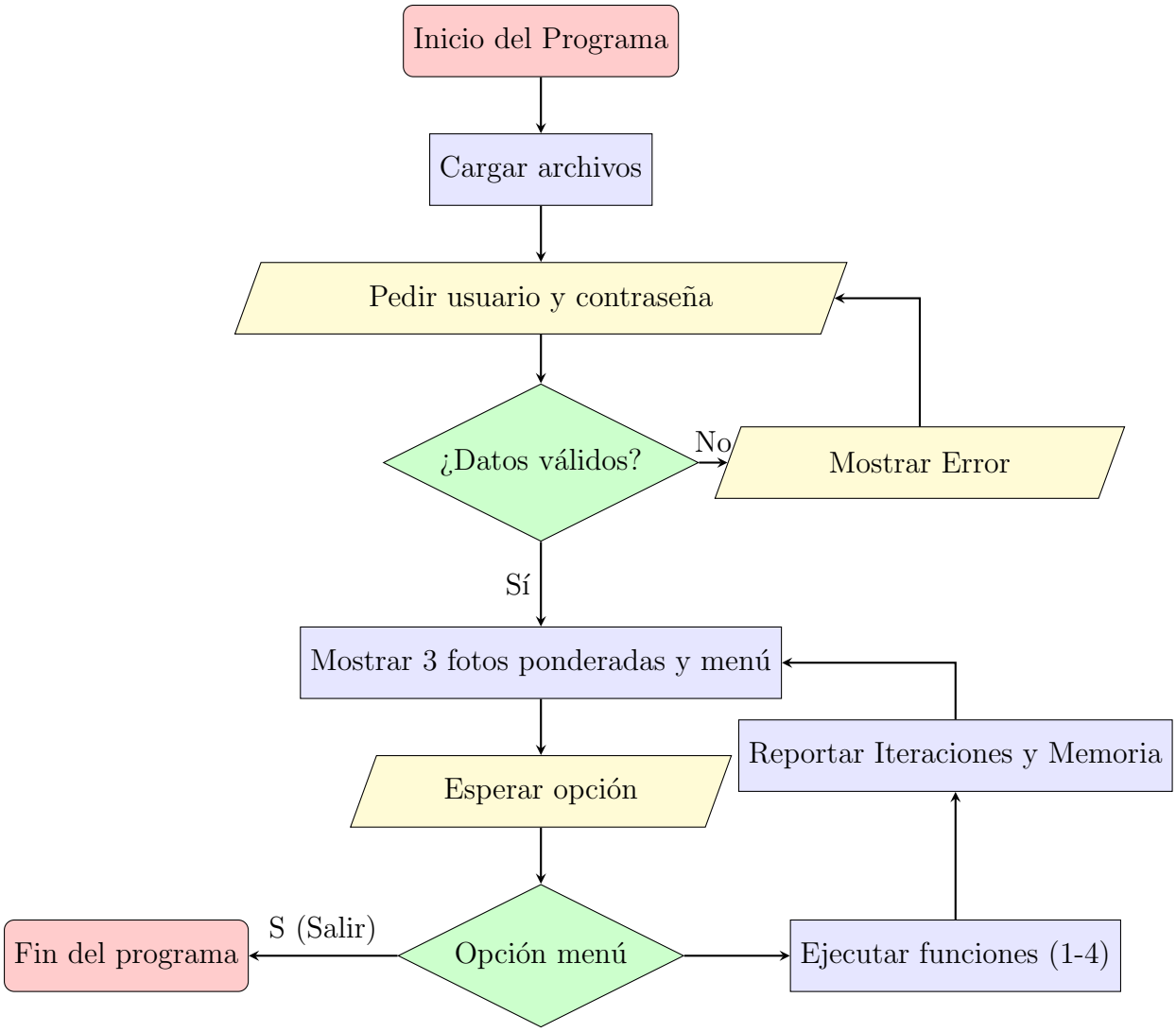
El almacenamiento no se usará como área de procesamiento temporal. Al iniciar, el programa cargará todo. Hemos planteado el siguiente formato para facilitar la lectura de cadenas.

Ejemplo estructural:

- `usuarios.txt`: `username|AAAAMMDD|correo|contrasena|ciudad|pais|AAAAMMDD`
- `amistades.txt`: `idUsuarioA|idUsuarioB` (Se lee y alimenta la matriz).
- `fotos.txt`: `idAlbum|nombreFoto|AAAAMMDD|tamanoBytes|likes`

3. Diagrama de Flujo: Ciclo Principal del Programa

El siguiente diagrama detalla el paso a paso lógico general, desde la inicialización hasta la interacción con el usuario en consola.



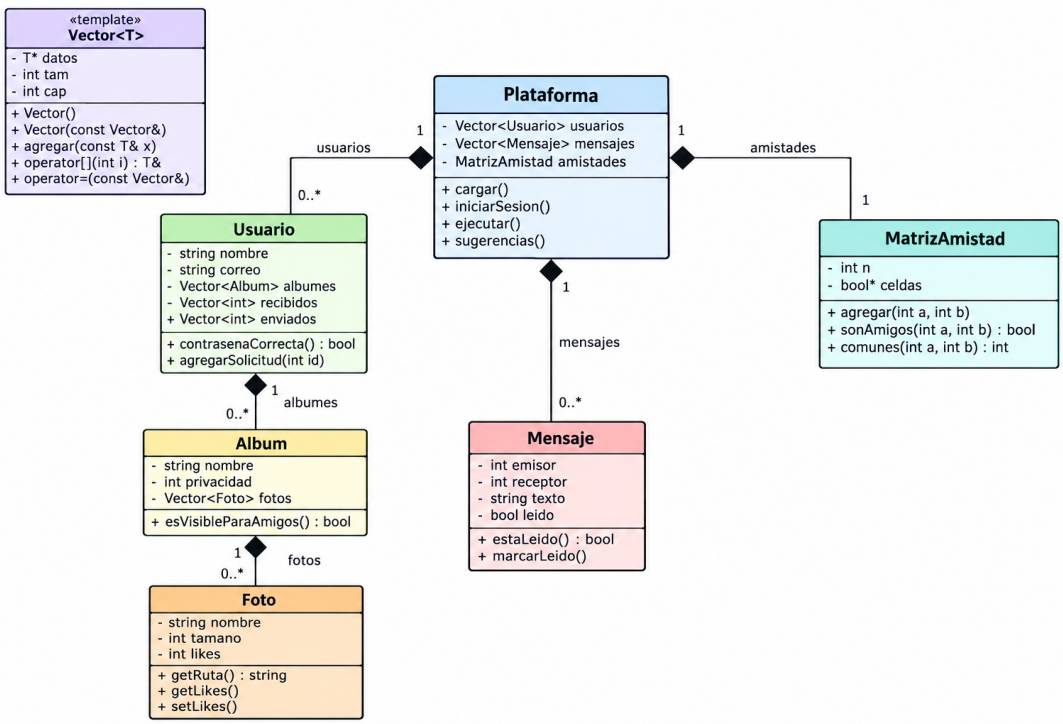


Figura 1: Diagrama de clases propuesto

4. Descripción general de las clases:

- **Plataforma:** Actúa como el cerebro del sistema. Es quien maneja la memoria global, ya que centraliza las listas de todos los usuarios, los mensajes y el registro de amistades. Se encarga de cargar los datos iniciales, gestionar el inicio de sesión y coordinar las funcionalidades principales.

- **Vector$< T >$:** Su trabajo es gestionar arreglos dinámicos en memoria que crecen automáticamente. Se usa en todo el programa para agrupar elementos (listas de usuarios, fotos, álbumes, etc.) sin preocuparnos por un tamaño fijo inicial.

- **Usuario:** Representa el perfil de cada persona en la red social. Almacena sus datos básicos y actúa como contenedor de sus álbumes. Para ahorrar memoria, no guarda copias de los mensajes ni de sus amigos, sino que utiliza índices numéricos (referencias a las listas globales en Plataforma) para saber qué mensajes ha enviado o recibido.

- **MatrizAmistad:** Es la estructura encargada de mapear la red de contactos. Funciona matemáticamente como una cuadrícula que registra las conexiones mutuas. Permite consultar instantáneamente si dos personas son amigas y es clave para calcular los amigos en común al generar sugerencias.

- **Mensaje:** Modela la información de los textos que se envían los usuarios. Al existir en una lista global única, evita que el texto se duplique en la memoria del emisor y del receptor. Solo registra los índices de quién lo envía, quién lo recibe, su contenido y si ya fue leído.

- **Album:** Funciona como una carpeta organizadora dentro del perfil de un usuario.

Además, gestiona la privacidad, dictando si su contenido es público, privado o solo para amigos.

- **Foto:** Almacena metadatos como el tamaño, la cantidad de likes y el nombre del archivo. A partir de esta clase se ensambla dinámicamente la ruta absoluta en disco necesaria para "visualizar"la imagen.

4.1. Link del repositorio

`https://github.com/LuisC645/ude-book.git`